HNSW Indexing & Retrieval, and Migration Idempotency

Two separate things I learned and implemented this week, documented together since both came out of hardening the pipeline beyond "works in the demo": how HNSW actually works under the hood (not just "it's the index we switched to"), and how I made schema migrations safe to run from more than one replica at once.

Part 1: HNSW — Indexing and Retrieval

What it actually is

HNSW (Hierarchical Navigable Small World) is a graph-based approximate nearest-neighbor structure, not a clustering method like IVFFlat. Instead of grouping vectors into fixed buckets, it builds a multi-layer graph where each vector is a node, connected to a handful of its nearest neighbors.

Simplest way I found to think about it: like a highway system. The **top layer** is sparse — few nodes, long-range connections, like highways linking distant cities. The **bottom layer** is dense — every vector is present, connected to its close neighbors, like local streets connecting every building. A search starts at the top layer, jumps quickly toward the right general area using the sparse long-range links, then descends layer by layer, using denser and denser connections to home in on the actual nearest neighbors. Fast approximate global positioning first, precise local search last.

How indexing (insertion) works

Every time a new vector is inserted:

1. It's randomly assigned a "top layer" it will participate in — higher layers are exponentially less populated, so most vectors only exist in the bottom, densest layer, and only a few "get promoted" to participate in the sparse upper layers. This is what creates the highway/local-street structure.
2. Starting from an existing entry point, the algorithm searches for the node's nearest existing neighbors, descending layer by layer, and connects the new node to the best candidates found at each layer.

This is the core structural reason HNSW doesn't suffer from the problem IVFFlat had for us: there's no one-time clustering step computed from whatever data existed at build time. Every insertion connects into the graph based on the graph's *current* state, so a table that starts empty and fills up over time ends up structurally identical to one that was fully loaded before the index was built. (Full reasoning on why this mattered and what we compared it against — REINDEX, REINDEX CONCURRENTLY — is in a separate note; not repeating that decision trail here.)

How retrieval (search) works

A query vector enters at the top layer's entry point, greedily moves to whichever neighbor is closest at that layer, and drops down a layer once no closer neighbor is found at the current one. At the bottom (densest) layer, instead of stopping at the single closest node, it maintains a **candidate list** of size `ef_search` and explores that many nearby nodes before returning the top-k closest ones found. This is an approximation, not an exhaustive search — it's why HNSW search doesn't check every row, and why it can occasionally miss the true nearest neighbor if `ef_search` is set too low.

The three parameters, and what each actually controls

- **`m`** — the maximum number of connections each node keeps to other nodes, per layer. This defines the graph's actual density/richness. Higher `m` means more edges per node — better recall (more paths to find close neighbors), but more memory (every node stores more links) and somewhat slower traversal, permanently, since the graph itself is bigger. This is a structural property of the index, not something you can change without rebuilding.
- **`ef_construction`** — the size of the candidate list examined *while inserting* a node, when deciding which `m` neighbors are actually worth linking to. Higher `ef_construction` means a more thorough search for good

neighbors at insert time, producing a higher-quality graph for a given `m` — but it costs insert-time latency, and since our consumer inserts continuously, that cost is recurring, not one-time. Unlike `m`, it doesn't change the final graph's size, just how well-chosen its connections are.
- **`ef_search`** — the query-time equivalent of `ef_construction`: how many candidates get examined while traversing the graph to answer a specific search. Doesn't touch the graph at all, and can be changed instantly via `SET`, no rebuild required — the cheapest and most flexible of the three knobs to tune after the fact.

Rough analogy that helped me keep these straight: `m` is how many friends each person is allowed to have; `ef_construction` is how many candidates get interviewed before picking those friends; `ef_search` is how many friends-of-friends you're willing to ask when looking for someone specific.

Why higher isn't automatically better

`ef_construction` in particular is a recurring per-insert cost, not a one-time build cost, given continuous ingestion. The recall-vs-effort curve also isn't linear — it climbs fast early, then flattens hard. Pushing `ef_construction` from a few hundred into the thousands buys negligible additional recall while making every single insert to `events_raw` meaningfully slower, permanently. Worth tuning to where the curve flattens, not maximizing blindly.

How I actually validated the tuning, not just picked numbers

Ran the same index build multiple times at different settings (`m=24 / ef_construction=200`, then `m=32 / ef_construction=400`), and for each setting, ran 5 real test queries through two paths: HNSW's approximate search, and a full brute-force sequential scan (checks every row directly — slow, but always 100% correct, since nothing is skipped). Comparing HNSW's top-k results against brute-force's top-k results for the same query is a direct recall measurement — it answers "did the approximate index actually find what the exact search would have found," not just "does this look plausible." Kept raising `m`/`ef_construction` until HNSW's results matched brute-force reliably across all 5 queries, not just one — since HNSW's approximate search has inherent randomness, one lucky match isn't proof the setting is actually good.

Landed on `m = 32, ef_construction = 400` for the index build, with `ef_search` also raised at query time for the same reason — the defaults are tuned for datasets much larger than ours, and were occasionally returning a decent-but-not-actually-closest match, silently, with no error.

Part 2: Migration Idempotency — Locks and Ledger

The problem

Every consumer startup re-runs every `*.sql` migration file from scratch, relying entirely on `IF NOT EXISTS` guards inside the SQL for safety. That guard isn't atomic across concurrent sessions: if two replicas start at the same instant, both can check "does this table exist," both see "no," and both attempt to create it — one succeeds, one throws a duplicate-object error and crashes. There's a second, quieter problem underneath: `IF NOT EXISTS` only protects simple create-type statements. A future migration that isn't safely re-runnable — an `ALTER TABLE`, a data backfill — would either error or silently double-apply on every restart, since nothing was tracking which migrations had already completed.

Doesn't bite at today's single replica, but becomes a real startup crash loop the moment this scales to more than one consumer instance, or the moment a non-idempotent migration gets added.

The fix, two parts

1. An exclusive Postgres advisory lock, acquired before any migration runs. Advisory locks are session-level locks Postgres provides specifically for application-level coordination like this — they don't lock any table or row, they're just a named "only one session may hold this" flag. If a second replica starts at the same moment, it doesn't race the first — it blocks and waits until the lock is released, then proceeds safely once the first replica has already finished (and, per the ledger below, finds there's nothing left to do).

2. A ledger — a small tracking table recording which migration files have completed successfully, one row per file. Every startup checks this table first, before attempting each migration: "has this one already run? If so, skip it." This is what makes even a non-idempotent migration (an `ALTER TABLE`, a backfill) safe to leave in the startup path indefinitely — it only ever actually executes once, no matter how many times the process restarts or how many replicas exist.

How I verified it, not just assumed it worked

Started two copies of the migration process at the literal same instant against the same database, deliberately trying to trigger the race rather than trusting the fix in theory. Neither crashed — the second replica waited on the lock as expected — and the ledger table ended up with exactly one row per migration file afterward, not two, confirming no migration double-applied under real concurrent contention.